

Adapting A Vector-Symbolic Memory for Lisp ACT-R

Meera Ray (mfr5832@psu.edu)

Department of Computer Science and Engineering, The Pennsylvania State University
W209 Westgate Building, University Park, PA 16802. USA

Christopher L. Dancy (cdancy@psu.edu)

Department of Industrial and Manufacturing Engineering, The Pennsylvania State University
310 Leonhard Building, University Park, PA 16802. USA

Abstract

Holographic Declarative Memory (HDM) is a vector-symbolic alternative to ACT-R's Declarative Memory (DM) system that can bring advantages such as scalability and architecturally defined similarity between DM chunks. We adapted HDM to work with the most comprehensive and widely-used implementation of ACT-R (Lisp ACT-R) so extant ACT-R models designed with DM can be run with HDM without major changes. With this adaptation of HDM, we have developed vector-based versions of common ACT-R functions, set up a text processing pipeline to add the contents of large documents to ACT-R memory, and most significantly created a useful and novel mechanism to retrieve an entire chunk of memory based on a request using only vector representations of tokens. Preliminary results indicate that we can maintain vector-symbolic advantages of HDM (e.g., chunk recall without storing the actual chunk and other advantages with scaling) while also extending it so that previous ACT-R models may work with the system with little (or potentially no) modifications within the actual procedural and declarative memory portions of a model. As a part of iterative improvement of this newly translated holographic declarative memory module, we will continue to explore better time-context representations for vectors to improve the module's ability to reconstruct chunks during recall. To more fully test this translated HDM module, we also plan to develop decision-making models that use instance-based learning (IBL) theory, which is a useful application of HDM given the advantages of the system.

Keywords: ACT-R; vector-symbolic architectures; Lisp; distributional semantics

Introduction

In this paper, we describe our ongoing efforts to adapt HDM to work better with Lisp-based ACT-R, thus bridging a newer vector-symbolic model with decades of cognitive modeling done with Lisp ACT-R. Our eventual goal is to build a cognitive agent that simulates the actions taken by disaster survivors and the social factors that impact those decisions. We chose ACT-R due to the need for the cognitive agent to represent the cognitive processes of a disaster survivor well enough to accurately model the decisions they would make, relative to the empirical results gathered from human subjects; ACT-R has successfully modeled the decision making of humans in constrained experiments (Ritter, Tehranchi, & Oury, 2019), sometimes serving as a useful lower-level cognitive representation for rational level (i.e., Newell, 1990) decision-making theories (e.g., Gonzalez, Lerch, & Lebiere, 2003). At the same time, the model needs to include situational awareness to accurately model how different survivors will respond to the same disaster environment differently depending on

their social background (Prather et al., 2022). Here we use situational awareness to mean background knowledge, context, and worldview that puts the perceived environment into context (Lukosch, Bekebrede, Kurapati, & Lukosch, 2018). We hypothesize that a vector-symbolic memory architecture, Holographic Declarative Memory (Kelly, Arora, West, & Ritter, 2020), can be useful for representing this context by reading in related texts into memory, which in turn influences which chunks are retrieved at runtime and which procedures are run. Furthermore, the increased scalability of a system like HDM may be more appropriate for representing the impact of existing social structures, which may themselves be more latently represented in memory in a way that requires a large number of concepts and associations to adequately represent.

In the next sections, we first justify why it is worthwhile to use HDM when ACT-R already has a built-in Declarative Memory system. Next, we explain adaptations needed at the architectural level for (canonical, Lisp) ACT-R and HDM for integration. We conclude with a discussion of preliminary results of testing this new translated HDM and future work.

What Does HDM add to ACT-R?

Traditional ACT-R models have two main memory components for modelers to manipulate: procedural memory, typically production rules written by the developer, and declarative memory, which may store semantic or episodic information that is often entered by hand before the model runs (Ritter et al., 2019). ACT-R retrieves relevant memories based on environmental cues specified as requests in production rules. However, by default ACT-R's declarative memory (DM) system lacks partial memory recall, meaning a gradation of which chunks are relevant to a query, as opposed to returning either one matching chunk or none at all. Additionally, the time complexity to add and retrieve chunks as well as the space complexity to store chunks would be prohibitive to adding corpus-sized amounts of text to DM (Kelly et al., 2020). Holographic declarative memory (HDM) addresses these problems by representing each unique input token in vector space rather than storing the entire input, which allows for a continuous similarity measure between queries and memories. HDM can thus "read" large corpora of relevant text into a model's memory; it is thus a distributed semantics model like Word2vec made available as a memory system for

a cognitive architecture. Despite the machine-learning-like principle behind it, HDM still achieves high-level cognitive plausibility in free recall effects and significantly fits human performance in a decision task.

Thus, HDM presents a useful alternative to the default ACT-R declarative memory representation, especially for models that may need to represent sociocultural structures in memory and represent similarity between concepts or chunks in an architecturally-defined way. The latter ability is helpful for implementing theories in ACT-R that rely on context-dependent associations and similarities, such as Instance-Based Learning (IBL) Theory (Gonzalez et al., 2003). This vector-based representation may also make architecture-level integration with generative models (which typically also use vector-based representations for text-based “tokens”) more straightforward (e.g., see Dancy & Workman, 2023).

Fortunately, the implementations of both HDM and the newest (Lisp) ACT-R provide enough of a foundation for a connection between the two systems. Nonetheless, to integrate HDM into the canonical version of ACT-R, both systems need to be adapted. This need for adaptation and expansion of HDM is especially true if one wants to use HDM while keeping declarative memory use patterns that are common amongst ACT-R modelers.

Adapting ACT-R for HDM

We used the interface provided in the canonical ACT-R 7 Python connection files (ACT-R Research Group, 2023) and the HDM code Dr. Kelly wrote for Python-ACTR (Kelly, 2020) to implement the Lisp-side ACT-R commands with Python code. At present, we have implemented the core commands for adding to and retrieving from memory as well as convenience and utility commands like (*sgp*) and (*dm*). The command (*dm*) now prints the unique values stored by HDM rather than chunks, which are not explicitly stored, as well as a two-dimensional visualization of the HDM vector space. Goal chunks, which require more precise recall and typically do not come in a large scale number of chunks, are to be created with (*define-chunk*) rather than (*add-dm*) so ACT-R’s default systems are used rather than HDM. The parameters for the whole-chunk recall mechanism, which will be explored in the next section, can be set globally like any other parameter in (*sgp*) or locally in any memory retrieval request.

Adding Text to HDM in ACT-R

We added the ACT-R command (*preprocess-text*), which takes a raw plain text file and outputs a file ready to be added to memory. The preprocessing step uses the Natural Language Toolkit (NLTK) to remove *stopwords*, often-appearing words such as “and” or “the,” which don’t convey information specific to a task (Sarica & Luo, 2021).

We made the decision to read the text into HDM sentence by sentence, though (Kelly et al., 2020) don’t indicate whether or not this is necessary. When HDM encodes a list of values, it stores associations between all pairs of words

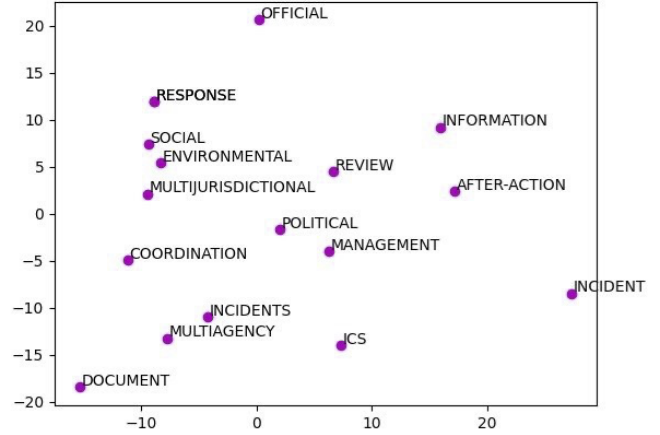


Figure 1: PCA Plot of HDM Vectors

in left-to-right order. For words that are far apart from each other, it is less worth computational resources to encode their associations with each other. Other ways of splitting the text may be optimal, but we chose the sentence as the unit to add at a time. Therefore, the preprocessing step tokenizes the text into sentences using NLTK’s Punkt pretrained tokenizer to find sentence boundaries.

From the preprocessed file, the modeler uses the (*read-corpus-hdm*) command to read in the preprocessed file to HDM. As implemented, this step can only take place after the model has been loaded. An example of a plot of HDM vectors after being added to memory is Figure 1. The corpus here is from a document instructing U.S. officials on how disaster response and recovery are organized.

Adapting HDM for ACT-R

A downside of HDM is that, unlike ACT-R DM, it lacks a mechanism to retrieve a chunk, a group of memories bound together, in its entirety without knowing the slot corresponding to each unknown value being requested. For example, a chunk “homeowner:yes damage:severe renter:no neighborhood:Eastville” could represent the status of a disaster survivor agent’s home after a flood; note the format of slot:value pairs. A cue to retrieve the chunk in DM would simply include a unique slot:value pair of the chunk — “homeowner:yes”. With HDM, one would have to specify that pair and then ask for one unknown at a time — “homeowner:yes damage:?”, “homeowner:yes renter:?” and so on. We extended the HDM implementation using a simple chaining method so multiple unknowns can be requested, but every slot in the chunk still needs to be specified; this is cumbersome for a modeller compared to ACT-R’s original DM. For us, it would require us to write more and longer production rules for our cognitive agent. Thus, we devised a method of full-chunk retrieval without storing chunks outright or deviating from HDM’s memory vector approach.

We define the problem as follows: after adding chunk

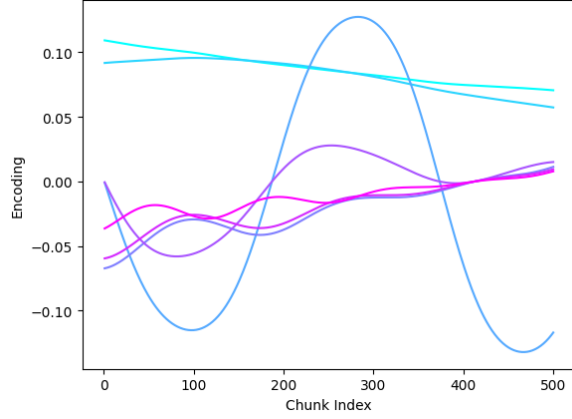


Figure 2: Oscillator function vector $\mathbf{T}(t)$ as a function of chunk index t . The functions plotted above are $T_1(t), T_{51}(t), T_{101}(t), T_{151}(t), T_{201}(t), T_{251}(t), T_{301}(t)$ out of the $\mathbf{T}(t) = [T_1(t), T_2(t), \dots, T_{320}(t)]$. Note how the oscillators vary from lower to higher frequencies. While some individual functions repeat in encoding value over the chunk index range, the vector at any one chunk index is unique.

$\mathbf{c} = \{s_1:v_1, s_2:v_2, \dots, s_n:v_n\}$ into memory, retrieve the entire chunk with a cue $\mathbf{q} = \{s'_1:v'_1, s'_2:v'_2, \dots, s'_m:v'_m\}$, where $\mathbf{q} \subseteq \mathbf{c}$ and the slots and their associated values are $s_1 \dots s_n$ and $v_1 \dots v_n$ respectively.

Time Encodings

First, we needed a way to represent which bits of memory were entered together as a chunk at the same time. We turned to neural oscillator encodings of serial memory (Brown, Preece, & Hulme, 2000) for a biologically-plausible theory of temporal encoding. In their model, there are fifteen oscillator functions $O_1(t) \dots O_{15}(t)$ of timestep t which range from lower to higher frequency, with R and ϕ as sources of noise:

$$O_i = \sin(\phi + t\theta_i) \quad (1)$$

$$\theta_i = SR2^i \quad (2)$$

$$R \sim \mathcal{N}(0, \sigma^2) \quad (3)$$

$$\phi \sim \mathcal{U}(0, \pi/\theta) \quad (4)$$

S is a scaling parameter that can be swapped out for different time scales. Brown et al. (2000) sets $S = 10^{-5}$, but we've found different values work better on different time scales (e.g. number of chunks). For the variance σ^2 of R , $\sigma^2 = 1$ in the Brown model but can be increased or decreased to increase or decrease the amount of noise.

The purpose of having noisy representations rather than simply, for example, sequentially numbering each chunk index, is to model the recall of human subjects in serial order memory tasks. An incorrect item is more likely to be exchanged for a correct item during recall the closer the two

items are in position. When positions are hierarchical, e.g. lists of lists, often the position of an item within a list is recalled correctly even if the list's position compared to other lists is not. For example, one may recall an item being in the third value of the second chunk instead of the third value in the first chunk. Multiple oscillating functions represent this hierarchical error by creating "bumps" in similarity between cognitive representations of two items even as their similarities decrease overall as their positions grow further apart.

To work more simply within the HDM framework, we flattened Brown et al. (2000)'s sixteen "learning context" vectors of twenty elements each into a single 320-element time vector $\mathbf{T}$ for each timestep. Each element T_i of the time vector in Equation 5 randomly draws four of the 15 oscillators O_j from Equation 1, with cosine being randomly substituted for sine with a probability of half:

$$T_i = \prod_{j=1}^4 \sin(O_j) \quad (5)$$

The function $\mathbf{T}(t) = [T_1(t) \dots T_{320}(t)]$ is generated and stored when the model is loaded. For each chunk added to memory, the time step t increases by 1 and the time vector $\mathbf{T}(t)$ is calculated. Note that the four oscillator functions randomly chosen above are not chosen with uniform probability; they're selected so the low-frequency oscillators make up a larger share than the higher-frequency oscillators. We did this because we found too many high-frequency oscillators led to nearly-repeating vectors over a small period of time. The probability distribution we selected was a discretized and bounded modification to the exponential distribution. We found it to be the best match for how the oscillators are distributed in Figure 6 of Brown et al. (2000). The best match for the figure is with a scaling parameter of $\beta = 5.125$. However, the distribution can be used with different scaling parameters for learning context vectors of arbitrarily chosen sizes if one adjusts the scaling parameter to be the expected value of which oscillators is chosen. For example, the scaling parameter β being 5.125 means the center of the probability mass is between the sixth and seventh oscillator (zero indexing). A plot of an oscillator vector is shown in Figure 2. We used the default parameters described earlier ($\sigma^2 = 1, \beta = 5.125$) but set the time scaling parameter $R = 5 * 10^{-6}$ to reflect the large range of chunk indices from 1 to 500.

Time HDM Vectors

Now that we have a continuous time representation, we need to somehow bind the representation with HDM's memory vectors. HDM itself uses the holographic reduced representations (HRR) framework (Plate, 1995). Plate shows how memory traces can be constructed by binding together two "holographic" vectors $A, B \in \mathbb{R}^n$ using circular convolution ($\circledast$). From a memory trace $C = A \circledast B + \dots$ and a cue A , a noisy version of B can be recovered. Simply using the time vector itself as a holographic vector would violate HRR's assumption that bound vectors are drawn from the same distribution.

HDM’s holographic vectors are simply drawn from the normal distribution, while the time vectors are oscillating outputs of $\mathbf{T}(t)$. Instead, we needed to treat the time vector as a kind of value being stored. The problem is that HDM, like ACT-R, works with discrete memory values even if it encodes them with continuous values. To encode the continuous time vector, we adopted the fractional binding operation from Komer, Stewart, Voelker, and Eliasmith (2019) and composed it with $\otimes$ from Plate to arrive at a holographic representation $\tilde{\mathbf{T}}$ of the time vector $\mathbf{T}$.

$$\tilde{\mathbf{T}} = \mathcal{F}^{-1} \left\{ \sum_{l=1}^{320} \mathcal{F}(e_{t_l}) \mathbf{T}(t) \right\} \quad (6)$$

The function $\mathcal{F}$ above is the Fourier transform. The HRR vectors $e_{t_1} \dots e_{t_{320}}$ are generated from the same distribution as HDM’s environment holographic vectors. Next, we associate the chunk’s time vector with each bit of memory in the chunk. We store these associations in mt vectors – Each HDM memory vector m has an associated time-memory vector mt . This does double the memory usage but keeps the same space complexity class. When a chunk $s_1 : v_1, \dots, s_n : v_n$ is added to memory, a $\tilde{\mathbf{T}}$ is created for it as per Equation 6. The mts are updated with the following rule:

$$mt \leftarrow mt + \tilde{\mathbf{T}} \otimes \sum_{c=1}^n s_c \otimes v_c \quad (7)$$

where s_c and v_c are corresponding slot-value pairs in a chunk.

This update rule, unlike the update rule for HDM’s memory vectors, does not add noise. From an implementation standpoint, adding noise would interfere with the recalling of added traces in Plate (1995), which already has some noise in reconstruction. From a cognitive processes representation standpoint, the noise to represent cognitively plausible time-memory errors is already represented in the time vectors themselves as discussed earlier.

To retrieve the chunk $\mathbf{c}$ from cue $\mathbf{q} = \{s'_1 : v'_1, s'_2 : v'_2, \dots, s'_m : v'_m\}$, we construct HRR Q from $\mathbf{q}$, take its inverse, and bind it with each memory-time vector mt to obtain a noisy reconstruction $\hat{\mathbf{T}}$ of the time HRR:

$$Q = \sum_{c=1}^m s'_c \otimes v'_c \quad (8)$$

$$\hat{\mathbf{T}} = mt \otimes Q^{-1} \quad (9)$$

The retrieval works because $\otimes$ is distributive, so mt contains $\tilde{\mathbf{T}} \otimes s_c \otimes v_c$ for each slot:value pair. Note that the inverse operation used here is the approximate inverse from Section VII, Subsection C in (Plate, 1995).

We are left with $\hat{\mathbf{T}}_1 \dots \hat{\mathbf{T}}_N$, the collection of reconstructed time vectors for each of the N mts . As per Brown et al. (2000), we step through the time vector function $\mathbf{T}(t)$ from $t = 1$ to the current time step in the model, comparing at each time step which of the reconstructed time vectors $\hat{\mathbf{T}}$ have the

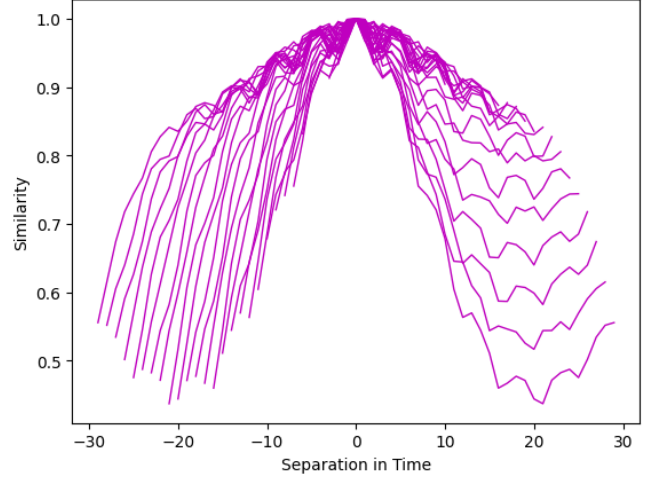


Figure 3: The dot product similarity between our time vector $\mathbf{T}(t') \cdot \mathbf{T}(t'')$ (y axis) and $t' - t''$ on the x axis. It peaks at 1 (comparing identical vectors) and has smaller local maxima to represent noise in serial memory recall.

greatest similarity with the current $\mathbf{T}(t)$. Depending on the method chosen — taking the top p results, all results above a threshold, or results meeting the previous two criteria — we end up with a list of possible slots and values making up a chunk. HDM tracks which memory vectors are slots, so we keep the mts corresponding to slots and then build a complete cue $\mathbf{q} = \{s_1 : ? \dots s_n : ?\}$ that contains every slot. The chunk can then be requested using multiple unknown request chaining, mentioned at the beginning of this section, with HDM’s built-in retrieval mechanism. Thus, we are in principle able to retrieve the entire chunk.

It’s worth noting that, like HDM, the time complexity scales with the number of unique values rather than the total number of chunks. Still, an option to only turn on slot pulling encoding for chunks of interest so only a subset of relevant time-memory vectors are stored would reduce the retrieval times. However, remembering chunks (e.g. sentences) stored during the corpus adding step could be useful for some modeller applications as well.

Preliminary Results and Discussion

Time Encodings

We generated two time vector functions using exactly the same set of parameters: $\sigma^2 = 1$ for Equation 3, $S = 10^{-5}$ for Equation 2, and $\beta = 5.125$ as the exponential distribution scaling parameter to sample four oscillators from the 15 oscillator functions to generate each element in Equation 5. The time vector has 320 elements as per our adaptation of Brown et al. (2000).

In order to evaluate the quality of the time vector representations, we plotted a similarity function $\mathbf{T}(t') \cdot \mathbf{T}(t'')$ for all $t', t'' \in \{1, 2, \dots, 30\}$. The similarity function $\cdot$ is the dot product, which is equivalent to cosine similarity here since

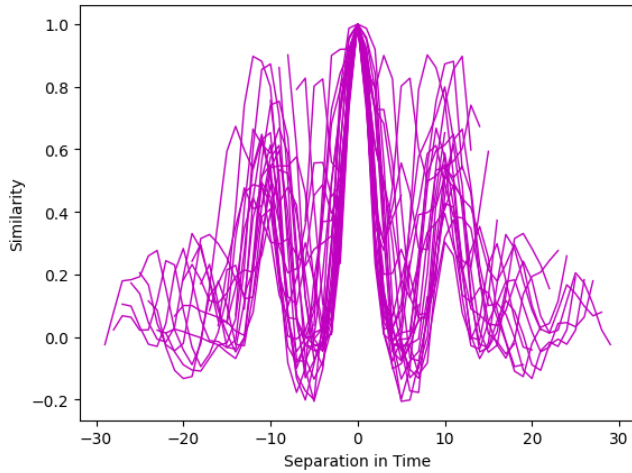


Figure 4: A self-similarity graph like Fig. 3 but with much larger “bumps,” indicating a noisier representation of serial memory.

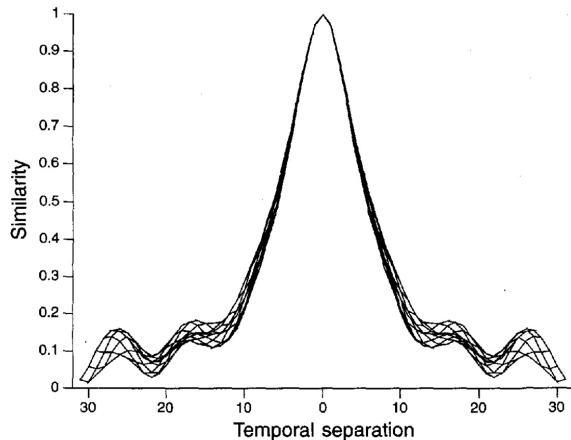


Figure 5: The (averaged) self-similarity of time vectors in Brown et al. (2000). Plot reproduced from Figure 7B of (Brown et al., 2000)

the vectors are normalized. The similarity plots are shown in Figure 3 and Figure 4.

As noted earlier, a similarity function between time vectors taken at subsequent time steps should show two properties: (1) a gradual decrease as the times grow further apart and (2) some oscillation to represent the cognitive error that arises from time’s hierarchical representations. Our time encodings do show both of these properties, but the “bumps” that represent the error are more pronounced than in self-similarity plot of Brown et al. (2000), reproduced here in Figure 5. Both plots are over the same time step range and equal corresponding parameters as our plot. A possible explanation is that the dot product between two large vectors rather than the average dot product between two sets of smaller vectors results in more noise, which Brown et al. (2000) mentioned as a justification for his choice of the latter representation.

Furthermore, one can observe a stark difference between the self-similarity of Figure 3 and Figure 4 despite having oscillator parameters generated from the same probability distributions with the same parameters. In our tests, neither increasing nor decreasing σ^2 had much consistent effect on how much variability appeared between the self-similarity plots of time vector functions. Increasing or decreasing the exponential distribution scale parameter β to respectively favor or disfavor slower oscillators had surprisingly little effect either on the variance between time functions or on the level of noise any particular time function showed in self-similarity plots. Further study is needed to quantify the self-similarity noise in a single comparable metric and decide which parameters yield representations that best fit observed cognitive effects in serial order.

Conclusion

So far, we have described how full-chunk recall can be supported without needing to store the chunks themselves nor a data structure scaling in size with the number of associations between values in the chunk. The time vector representations we have implemented so far are noisy but still provide distinct representations for distinct chunks. Work to measure slot pulling effectiveness across a wider variety of models is ongoing.

In the future, we want to try another time encoding for associating chunks together besides neural oscillators. We also want to examine different chunk retrieval mechanisms. So far, we take the two inputs —reconstructed time vectors per time-memory vector and the time vectors themselves— and compute the similarity between them as the only ranking criterion. However, we should also expect that the time vector reconstructions associated with the same chunk ought to cluster together in vector space. Perhaps the true time vectors represent cluster centers and a clustering measure could aid accurate and precise retrieval.

The vector-symbolic approach used here also presents an opportunity for more straightforward considerations of ways to integrate generative models within ACT-R. Such an inte-

gration would hold implications for expanded sociocultural representations in a cognitive architecture like ACT-R, as argued by Dancy and Workman (2023). As previously mentioned, having these sociocultural representations in memory would also give an opportunity to explore the impacts of sociocultural structures on decision making behavior given the interdependence between memory and decisions. We plan to explore interactions between sociocultural structures, memory, and decision making using HDM (and eventually generative models) in the context of IBL.

Adequately representing these sociocultural structures in memory requires memory representations to be scaled up efficiently. At the same time, there is value in making production systems work reliably with vector memory; production rules give explainable reasoning for decision making that generative models often struggle with. Full-chunk recall is needed to support flexible, reliable production systems interacting with vector-based memory. Also important is that full-chunk recall makes sure memories specific to an agent rather than its broader worldview are used at the appropriate times during decision making. The full-chunk recall mechanism and text-to-memory pipeline explained here hold promise for computational cognitive architectures like ACT-R to model the cognitive role that social structures play in human decision making.

References

- ACT-R Research Group. (2023, July). *ACT-R ACT-R 7.27.9 software*. Retrieved 2023-12-06, from <https://act-r.psy.cmu.edu/act-r-7-27-9-software/>
- Brown, G. D. A., Preece, T., & Hulme, C. (2000). Oscillator-Based Memory for Serial Order. *Psychological Review*, 107(1), 127–181.
- Dancy, C. L., & Workman, D. (2023). On integrating generative models into cognitive architectures for improved computational sociocultural representations [Conference Proceedings]. In *Aaai fall symposium series* (Vol. 2, p. 256–261). Washington, DC: AAAI Press.
- Gonzalez, C., Lerch, J. F., & Lebiere, C. (2003). Instance-based learning in dynamic decision making [Journal Article]. *Cognitive Science*, 27(4), 591–635. doi: 10.1207/s15516709cog2704.2
- Kelly, M. A. (2020, July). *ecphory/HDM*. Retrieved 2023-09-25, from <https://github.com/ecphory/HDM> (original-date: 2020-03-29T22:27:50Z)
- Kelly, M. A., Arora, N., West, R. L., & Reitter, D. (2020). Holographic Declarative Memory: Distributional Semantics as the Architecture of Memory. *Cognitive Science*, 44(11), e12904. doi: 10.1111/cogs.12904
- Komer, B., Stewart, T. C., Voelker, A. R., & Elias Smith, C. (2019, July). A neural representation of continuous space using fractional binding. Montreal, Canada.
- Lukosch, H. K., Bekebrede, G., Kurapati, S., & Lukosch, S. G. (2018, June). A Scientific Foundation of Simulation Games for the Analysis and Design of Complex Systems. *Simulation & Gaming*, 49(3), 279–314. doi: 10.1177/1046878118768858
- Newell, A. (1990). *Unified theories of cognition* [Book]. Cambridge, MA, USA: Harvard University Press.
- Plate, T. (1995, May). Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3), 623–641. doi: 10.1109/72.377968
- Prather, R. W., Benitez, V. L., Brooks, L. K., Dancy, C. L., Dilworth-Bart, J., Dutra, N. B., ... Thomas, A. K. (2022). What Can Cognitive Science Do for People? *Cognitive Science*, 46(6), e13167. doi: 10.1111/cogs.13167
- Ritter, F. E., Tehranchi, F., & Oury, J. D. (2019). ACT-R: A cognitive architecture for modeling cognition. *WIREs Cognitive Science*, 10(3), e1488. doi: 10.1002/wcs.1488
- Sarica, S., & Luo, J. (2021, August). Stopwords in technical language processing. *PLOS ONE*, 16(8), e0254937. (Publisher: Public Library of Science) doi: 10.1371/journal.pone.0254937